

4.1 Core

Contains the main logic of the application.

Examples:

- Command processing
- Task planning
- Execution management
- Tool/feature registration

The core should not contain application-specific implementation when that functionality belongs in a plugin.

4.2 Plugins

Each plugin should provide a specific group of capabilities.

For example:

```
plugins/  
└─ file_manager/  
    ├── plugin.py  
    ├── operations.py  
    └─ tests/
```

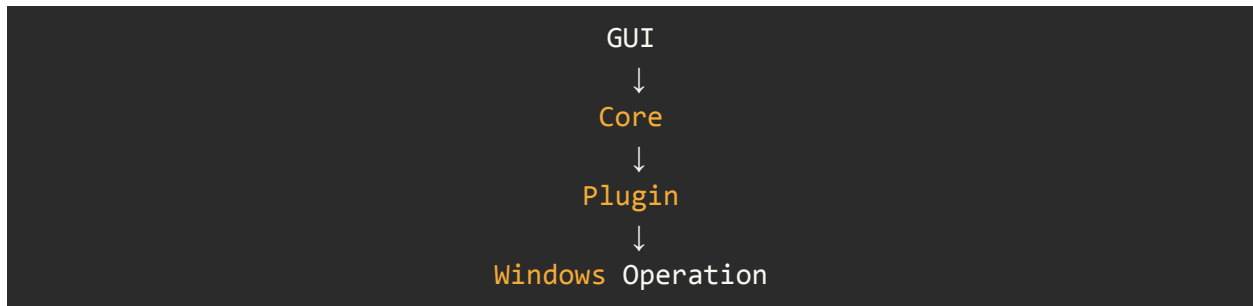
Plugins should communicate with the core through defined interfaces rather than directly modifying core components.

4.3 GUI

Contains only presentation and user-interaction logic.

The GUI should not directly implement file operations, process management, or AI logic.

For example:



rather than:



4.4 Tests

Tests should be separated from production code.

At minimum:

```
tests/
├─ unit/
├─ integration/
└─ end_to_end/
```

- **Unit tests:** Test individual functions or classes.
- **Integration tests:** Test interaction between modules.
- **End-to-end tests:** Test complete user workflows.

4.5 Documentation

Project documentation should be stored separately from source code.

Examples:

```
docs/  
├── architecture.md  
├── plugin_guide.md  
├── coding_standards.md  
└── testing.md
```

The coding standards document itself should therefore live under `docs/`.